CRISiS Lab | SLIIT

Earthquake Early Warning Sensor

Startup & Configuration Guide

Version 2.0

June 2026

Developed at CRISiS Lab, New Zealand & SLIIT, Sri Lanka

GitHub: gssamuditha/EEW-Sensor-Firmware-and-UI

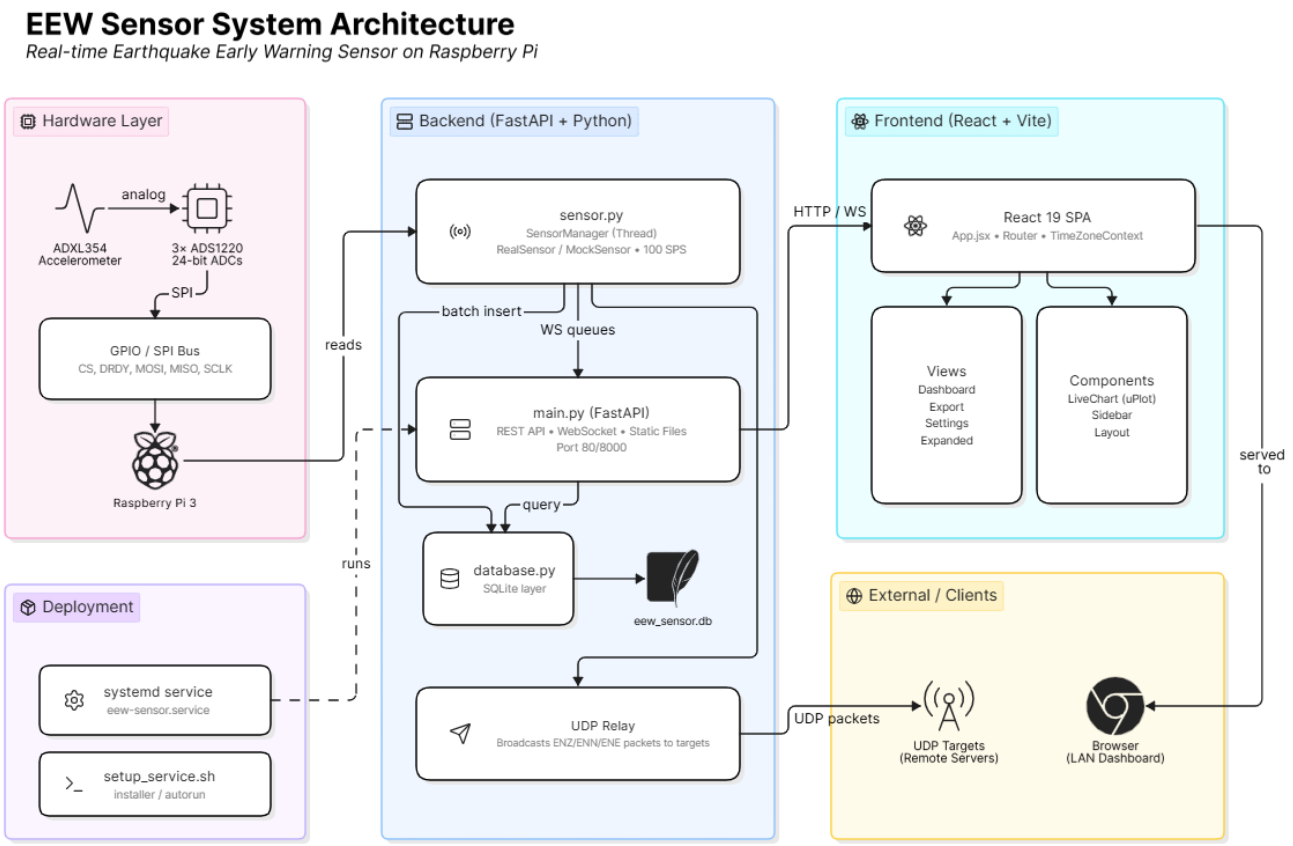
1. Project Overview

The EEW Sensor is a real-time Earthquake Early Warning system built for deployment on a Raspberry Pi single-board computer. It reads triaxial acceleration data from an ADXL354 MEMS accelerometer through three 24-bit ADS1220 ADCs connected via SPI, streams live data over WebSocket to a browser-based React dashboard, stores readings in a local SQLite database, and can relay data in real time to remote monitoring servers over UDP.

This guide covers everything needed to get the sensor hardware wired, the software installed, the service configured to auto-start on power-up, and the web dashboard accessible on your local network.

1.1 System Architecture

The diagram below shows how the hardware sensor, backend firmware, and browser dashboard communicate with one another.



1.2 Key Features

- 100 samples-per-second triaxial acceleration streaming (ENZ / ENN / ENE)
- Live waveform dashboard accessible from any device on the same network
- Automatic zero-level calibration on startup
- 24-hour rolling SQLite database with CSV export
- UDP data relay to multiple remote servers simultaneously
- Single-binary deployment — React UI served directly by FastAPI
- Auto-start on boot via systemd — no keyboard or monitor required
- Accessible via friendly URL: http://cl.local (no IP address needed)

1.3 Hardware Requirements

Component	Specification / Notes
Single-Board Computer	Raspberry Pi 3 Model B
Power Supply	Official Raspberry Pi USB-C 5 V / 3 A power supply
Storage	8 GB+ microSD card — Class 10 or A2 speed rating recommended
Network	Ethernet cable or Wi-Fi for dashboard access from other devices
Operating System	Raspberry Pi OS Lite (64-bit)

1.4 Software Stack

Component	Version / Detail
Python	3.9 or newer (runtime for sensor firmware and API server)
FastAPI + Uvicorn	ASGI web framework serving REST API, WebSocket, and static React UI
SQLite	Bundled with Python — stores sensor readings and configuration
psutil	System telemetry (CPU %, disk usage, uptime, IP address)
Node.js	18 or newer — required only to build the React frontend
React 19 + Vite	Browser dashboard with real-time charts
uPlot	High-performance WebGL time-series chart library
Tailwind CSS v4	Utility-first styling framework
React-Leaflet	Interactive map for device location configuration

2. Hardware Setup & Wiring

2.1 GPIO Pin Mapping

The ADXL354 accelerometer connects to the Raspberry Pi through three ADS1220 24-bit ADC chips, one for each axis. All three ADCs share the same SPI data lines (MOSI, MISO, SCLK) but each has its own Chip Select (CS) and Data Ready (DRDY) line. Pin numbers below use BOARD (physical connector) numbering.

GPIO Pin (BOARD)	Function	Connected To
Pin 19	SPI MOSI	ADC DIN (all 3 ADCs)
Pin 21	SPI MISO	ADC DOUT (all 3 ADCs)
Pin 23	SPI SCLK	ADC SCLK (all 3 ADCs)
Pin 35	CS – Z Axis	ADS1220 #0 (Z) Chip Select
Pin 33	CS – X Axis	ADS1220 #1 (X) Chip Select
Pin 36	CS – Y Axis	ADS1220 #2 (Y) Chip Select
Pin 11	DRDY – Z	ADS1220 #0 Data Ready
Pin 15	DRDY – X	ADS1220 #1 Data Ready
Pin 13	DRDY – Y	ADS1220 #2 Data Ready
Pin 16	Self-Test 1	ADXL354 ST1
Pin 18	Self-Test 2	ADXL354 ST2
Pin 22	Standby	ADXL354 STBY

⚠ **WARNING** All connections must be made with the Raspberry Pi powered off

2.2 Sensor Parameters (Default Values)

The following parameters are set in backend/sensor.py and reflect the physical characteristics of the ADXL354 and ADS1220 hardware. Only change these if you are using a different hardware variant.

Parameter	Default Value
ADC Reference Voltage (VREF_ADCS)	1.8 V per axis
ADXL354 Sensitivity (ACC_SENSITIVITY_V_PER_G)	0.4 V/g
ADC Full-Scale Code (FULL_SCALE)	8,388,607 ($2^{23} - 1$ for 24-bit)
Default Calibration Duration	60 seconds (configurable via Settings page)
UDP Samples Per Packet (SAMPLES_PER_PACKET)	25 samples

3. Raspberry Pi OS Installation

This section walks through installing the operating system on the Raspberry Pi and enabling the SPI interface that the sensor hardware requires. You will need a Windows, Mac, or Linux computer to flash the microSD card.

3.1 Flash the Operating System

1	**Download Raspberry Pi Imager** Go to raspberrypi.com/software and download the Raspberry Pi Imager application for your computer's operating system (Windows, macOS, or Ubuntu).
2	**Choose the OS** Open Raspberry Pi Imager. Click 'Choose OS', select 'Raspberry Pi OS (other)', then choose 'Raspberry Pi OS Lite (64-bit)'. The Lite edition has no desktop environment — this is correct for headless sensor deployment.
3	**Configure settings before flashing** Click the gear icon (⚙) or press Ctrl+Shift+X to open the advanced options panel. Fill in: Hostname = cl, enable SSH, set username = crisislab (or your preferred name), set a strong password, and optionally configure your Wi-Fi network name and password.
4	**Flash the card** Insert your microSD card (8 GB minimum), click 'Choose Storage', select your card, then click 'Write'. Wait for the flash and verification to complete — this takes 3–5 minutes.
5	**First boot** Insert the microSD card into the Raspberry Pi and connect power. Wait 60–90 seconds for the first boot to complete, then SSH into the Pi from another computer on the same network.

ℹ INFO	If you set the hostname to 'cl' in the Imager settings, you can SSH using: ssh crisislab@cl.local — no need to look up the IP address.

3.2 Enable the SPI Interface

The ADS1220 ADCs communicate via SPI. This interface is disabled by default in Raspberry Pi OS and must be enabled before the sensor firmware can talk to the hardware.

Run raspi-config on the Pi:

```
sudo raspi-config
```

Navigate to: Interface Options → SPI → Enable → OK → Finish

Reboot the Pi to apply the change:

```
sudo reboot
```

After rebooting, verify that the SPI device files exist:

```
ls /dev/spidev*
# Expected output:
# /dev/spidev0.0  /dev/spidev0.1
```

> ⚠ **WARNING** If /dev/spidev* files do not appear after rebooting, the SPI interface was not saved correctly. Repeat the raspi-config steps above.

4. Software Installation

All commands in this section are run in terminal on the Raspberry Pi. The installation process takes approximately 10–15 minutes depending on internet speed.

4.1 Install System Dependencies

Install Python and git

```
sudo apt update
sudo apt-get install python3-dev build-essential
sudo apt install git -y
```

Install node

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 22
```

4.2 Clone the Repository

```
cd ~
git clone https://github.com/gssamuditha/EEW-Sensor-Firmware-and-UI.git
cd EEW-Sensor-Firmware-and-UI
```

4.3 Create the Python Virtual Environment

Create a Python virtual environment and install all backend dependencies:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r backend/requirements.txt
```

INFO	The requirements.txt file includes all needed packages: fastapi, uvicorn, websockets, pydantic, psutil, and RPi.GPIO / spidev for the Pi hardware interface.

4.4 Build the Frontend

The React dashboard must be compiled into static files before the FastAPI server can serve it. This step is only required on first installation and after any frontend code changes.

```
cd frontend
npm install
npm run build
cd ..
```

After the build completes, a frontend/dist/ folder will be created. The FastAPI backend serves these files directly — no separate frontend server is needed in production.

4.5 Verify the Installation (Manual Test)

Before setting up auto-start, test that the software runs correctly:

```
source .venv/bin/activate
cd backend
python main.py
```

Open a browser on any device on the same network and go to:

```
http://<pi-ip-address>:8000
```

The sensor will go through an initialisation sequence on first run:

1	**GPIO & SPI Init** Initialises GPIO pins and SPI bus. Takes about 1 second.
2	**Wake from Standby** Releases the ADXL354 from standby mode. Takes about 10 seconds.
3	**GPIO Settle** Allows GPIO voltages to stabilise. Takes about 1 second.
4	**Zero Calibration** Measures baseline offset with no motion. Duration is set by the Calibration Time setting (default 60 seconds). The dashboard will show 'Waiting for data...' during this phase.
5	**Live Streaming** Begins streaming at 100 samples/sec. Dashboard charts will update in real time.

5. Auto-Start on Boot (systemd)

Once you have confirmed that the software runs correctly, configure it to start automatically whenever the Raspberry Pi is powered on. The project includes a setup script that handles the entire systemd configuration.

5.1 Run the Setup Script

From the project root directory on the Raspberry Pi, run:

```
cd ~/EEW-Sensor-Firmware-and-UI
chmod +x setup_service.sh
./setup_service.sh
```

The setup_service.sh script automatically performs the following steps:

- Validates that it is being run from the correct project root directory
- Checks that the Python virtual environment (.venv) exists
- Fixes file ownership so the crisislab user can write to the SQLite database
- Creates /etc/systemd/system/eew-sensor.service with the correct paths
- Grants the CAP_NET_BIND_SERVICE capability (allows binding to port 80 without root)
- Configures Restart=always with a 5-second delay for automatic crash recovery
- Enables the service to start on every boot and starts it immediately

TIP After the script completes, the dashboard is accessible at http://cl.local — no port number needed. The service will restart automatically if the Pi loses power or the process crashes.

5.3 Managing the Service

Use the following commands to manage the running service:

Action	Command
Check if the service is running	sudo systemctl status eew-sensor.service
View live logs (real-time)	sudo journalctl -u eew-sensor.service -f
View the last 100 log lines	sudo journalctl -u eew-sensor.service -n 100
Stop the service	sudo systemctl stop eew-sensor.service
Restart the service	sudo systemctl restart eew-sensor.service
Disable auto-start on boot	sudo systemctl disable eew-sensor.service
Re-enable auto-start on boot	sudo systemctl enable eew-sensor.service

5.4 Set the Pi Hostname to cl

To access the dashboard using http://cl.local instead of an IP address, the Raspberry Pi's hostname must be set to 'cl'. This is done once via:

```
sudo hostnamectl set-hostname cl
sudo reboot
```

After the Pi reboots (allow 30–60 seconds), the dashboard is reachable from any device on the same network at:

```
http://cl.local
```

> **INFO** The .local address uses mDNS (Bonjour/Avahi). Windows and macOS support this natively. On some Linux machines, you may need to install avahi-daemon: sudo apt install avahi-daemon

6. Using the Dashboard

Once the service is running, open a browser on any device connected to the same network as the Raspberry Pi and navigate to http://cl.local. The dashboard has four main sections accessible from the left sidebar.

6.1 Dashboard — Live Telemetry

The main dashboard provides a single-screen view of all sensor data with no scrolling required. It is organised into a grid of widget panels:

Widget	Description
Device Details (left column)	Shows device name, model, active channels, local IP address, MAC address, GPS coordinates, and system uptime. The device name updates in real time when changed in Settings.
Network & Connections (top right)	Displays connection status indicators for Internet connectivity and the WebSocket data stream. Green badges indicate active connections.
Live Telemetry (centre)	Three stacked real-time waveform plots for ENZ (vertical), ENN (north), and ENE (east) axes. Charts use a 5-second rolling window with fixed-position X-axis grid lines. A green badge at the top shows the current samples/sec rate.
System Status (right)	Horizontal progress bars showing live CPU usage and disk usage, pulled from the Raspberry Pi via the /api/system_status endpoint every 5 seconds.
Server Actions (bottom right)	Quick-link buttons to Station View and Data View on the connected server platform.

6.2 Pause and Resume

A Pause button is located at the bottom-left of the telemetry chart area. Clicking it freezes the waveform display while continuing to buffer incoming data in the background. No samples are lost while paused. Clicking Resume immediately replays all buffered data so the chart catches up without gaps.

6.3 Expanded View

The View Expanded button (next to Pause) opens a dedicated full-screen chart page in a new browser tab. This page shows all channels stacked vertically at maximum width, with the sidebar and widget headers hidden to maximise waveform visibility. It is useful for detailed inspection of seismic events.

6.4 Data Export

Navigate to the Data Export page from the sidebar. Select a start and end date/time using the timezone-aware date pickers, then click Download CSV. The export queries the SQLite database for the selected time range and returns a CSV file with columns: timestamp, ENZ (m/s²), ENN (m/s²), ENE (m/s²).

6.5 Settings Page

The Settings page is organised into three tabs:

General Tab

- Device Name — Set a custom name for this sensor node (e.g. CRISIS-NODE-01). The name appears on the main dashboard.
- Device Location — Enter latitude and longitude manually, or click anywhere on the interactive map to place a pin and automatically populate the coordinates. The location is stored in the database and included in UDP relay packets.

Network Tab

- Wi-Fi Configuration — Shows the currently connected network name. Enter a new SSID and password and click Connect/Save to switch networks. Uses nmcli on the Raspberry Pi.
- Data Sharing — Master toggle to enable or disable UDP data forwarding. Below the toggle, manage the list of Data Cast IP targets. Each target requires a Name, IP Address, and Port. The sensor will relay data to all configured targets simultaneously.

System Tab

- Calibration Settings — Set the zero-level calibration duration in seconds. Recommended minimum is 60 seconds for stable baseline. Click Save Calibration to persist the value.
- Recalibrate — Stops the sensor loop and restarts it using the saved calibration time. The dashboard will show 'Waiting for data...' during the recalibration period.
- Restart Sensor — Triggers a full system reboot of the Raspberry Pi. A confirmation modal must be acknowledged before the restart proceeds, preventing accidental interruptions to live telemetry.

7. API Reference

The FastAPI backend exposes the following endpoints. All REST endpoints return JSON. The base URL in production is http://cl.local (port 80).

7.1 REST Endpoints

Method + Endpoint	Description	Request / Response
GET /api/settings	Retrieve all stored configuration	Returns JSON with targets, latitude, longitude, device_name, calibration_time, data_forwarding
POST /api/settings	Update configuration	JSON body: { targets, latitude, longitude, device_name, calibration_time, data_forwarding }
GET /api/system_status	System health metrics from the Pi	Returns cpu_percent, disk_used_gb, disk_total_gb, uptime_seconds, ip_address, mac_address, sps, internet_status
GET /api/export?start=&end=	Download sensor data as CSV	Query params: start and end as Unix epoch timestamps. Returns CSV file.
POST /api/sensor/recalibrate	Restart the sensor and recalibrate	Stops and restarts the sensor thread; uses calibration_time from the database
POST /api/wifi/connect	Connect to a new Wi-Fi network	JSON body: { ssid, password }. Runs nmcli on the Pi.
GET /api/wifi/status	Get currently connected SSID	Returns { connected: bool, ssid: string }
POST /api/system/restart	Reboot the Raspberry Pi	Runs sudo reboot. Requires prior confirmation in the UI modal.

7.2 WebSocket

Field	Detail
Endpoint	ws://cl.local/ws/stream
Protocol	JSON messages, one per sample group
Message format	{ t: float, ENZ: float, ENN: float, ENE: float }
Rate	100 messages per second (one per 10 ms sample interval)
Channel ENZ	Vertical axis acceleration in m/s²
Channel ENN	North axis acceleration in m/s²
Channel ENE	East axis acceleration in m/s²

7.3 UDP Relay Packet Format

When data forwarding is enabled and at least one Data Cast IP is configured, the sensor broadcasts UDP packets to all targets. Each packet contains a batch of 25 samples.

```
# Packet format (Python list, encoded as UTF-8 string):
['ENZ', timestamp_epoch_float, sample_1, sample_2, ..., sample_25]
['ENN', timestamp_epoch_float, sample_1, sample_2, ..., sample_25]
['ENE', timestamp_epoch_float, sample_1, sample_2, ..., sample_25]


# Example:
['ENZ', 1716924000.123, 0.021, 0.019, 0.023, ...]
```

To record UDP packets on a remote PC for verification, run the included utility script from the project root on the receiving machine:

```
python record_udp.py
# Optional flags:
python record_udp.py --port 2098 --output my_recording.csv
```

8. Development Setup (Windows / Mac / Linux)

For frontend and backend development on a regular computer (without Raspberry Pi hardware), the system includes a MockSensor that auto-activates on non-Pi machines, generating random acceleration data so the full UI can be developed and tested.

8.1 Clone and Install

```
git clone https://github.com/gssamuditha/EEW-Sensor-Firmware-and-UI.git
cd EEW-Sensor-Firmware-and-UI


# Backend
python -m venv .venv


# Windows:
.venv\Scripts\activate


# Linux / macOS:
source .venv/bin/activate


pip install fastapi uvicorn[standard] websockets pydantic psutil
```

```
# Frontend
cd frontend
npm install
```

8.2 Run in Development Mode

Open two terminal windows:

Terminal 1 — Backend:

```
cd backend
python main.py
# Server starts at http://localhost:8000
```

Terminal 2 — Frontend (hot reload):

```
cd frontend
npm run dev
# Vite dev server at http://localhost:5173
# API requests are proxied to :8000 automatically
```

Open http://localhost:5173 in your browser. The MockSensor generates random acceleration data so all charts and controls function identically to the real hardware deployment.

i INFO

The MockSensor is automatically selected when the code detects it is NOT running on a Raspberry Pi (Windows platform detected, or RPi.GPIO not installed). No configuration is needed.

9. Troubleshooting

9.1 Service Will Not Start

Check the service logs first:

```
sudo journalctl -u eew-sensor.service -n 50
```

Error Message	Likely Cause	Fix
attempt to write a readonly database	The eew_sensor.db file or backend/ directory is owned by root	Run: sudo chown -R crisislab:crisislab ~/EEW-Sensor-Firmware-and-UI/backend
No module named 'RPi'	RPi.GPIO is not installed in the virtual environment	Run: source .venv/bin/activate && pip install -r backend/requirements.txt
[Errno 98] Address already in use	Port 80 is already bound by another process	Run: sudo lsof -i :80 to find the process, then stop it
dist/ directory not found	Frontend was not built before starting the service	Run: cd frontend && npm run build
DRDY timeout on ADC 0	SPI not enabled or wiring error on ADC 0 DRDY pin	Run sudo raspi-config and re-enable SPI; verify pin 11 wiring

9.2 Dashboard Shows 'Waiting for data...'

- This is normal during the calibration phase. Wait the full calibration time (default 60 seconds) before expecting live data.
- Verify the service is running: sudo systemctl status eew-sensor.service
- Check the WebSocket connection in the browser Developer Tools → Network tab → filter by WS.
- If the service shows errors, check logs with: sudo journalctl -u eew-sensor.service -f

9.3 Cannot Access Dashboard from Another Device

- Confirm both devices are on the same Wi-Fi or Ethernet network.
- Try the IP address directly if .local does not resolve: hostname -I on the Pi gives you the IP.
- Verify the service is listening on port 80: ss -tlnp | grep 80
- On Windows, .local resolution requires Bonjour (installed with iTunes or Apple devices). If not available, use the IP address instead.

9.4 UDP Data Not Arriving at Remote PC

- Ensure the correct LAN IP of the receiving PC is entered in the Data Cast IPs list on the Settings → Network tab.
- Check the Data Forwarding master toggle is turned on.
- Windows Firewall may block incoming UDP on port 2098. Add an inbound rule allowing Python on UDP port 2098, or temporarily disable the firewall to test.

- Run python record_udp.py on the receiving PC to verify packets are arriving.
- Confirm the Pi and the receiving PC are on the same subnet (e.g. both 192.168.8.x).

9.5 Logo / Icons Not Displaying on Pi

This can occur when the production build is served and the Pi's FastAPI catchall route returns index.html for image requests instead of the actual image files.

```
# On the Pi, ensure you are using the updated main.py that includes
# explicit file serving before the SPA catchall route.
# Then rebuild and redeploy:
cd frontend && npm run build
sudo systemctl restart eew-sensor.service
```

9.6 Permission Denied Errors

```
# Reset ownership of the entire project directory:
sudo chown -R crisislab:crisislab ~/EEW-Sensor-Firmware-and-UI


# Ensure the backend directory is writable:
sudo chmod 755 ~/EEW-Sensor-Firmware-and-UI/backend


# Restart the service:
sudo systemctl restart eew-sensor.service
```

10. Project File Structure

10.1 Backend Module Summary

Module	Key Responsibilities
main.py	FastAPI lifespan management, REST API routes, WebSocket stream endpoint, static file serving, CORS, internet check, Wi-Fi nmcli commands, system reboot endpoint, sensor recalibrate endpoint
sensor.py	MockSensor (random data for dev), RealSensor (GPIO + SPI + ADC init + calibration + voltage-to-m/s² conversion), SensorManager (100 SPS acquisition loop, WebSocket broadcast, SQLite batching, UDP relay, SPS stats, thread safety with Lock)
database.py	init_db (creates sensor_data and settings tables with default values), insert_batch (bulk insert with context-managed connections), cleanup_old_data (24-hour retention), get_data_for_export (time-range query), get_settings / update_settings (JSON CRUD)

10.2 Frontend Module Summary

Module	Key Responsibilities
App.jsx	React Router v6 setup, wraps all views in the TimeZoneContext provider
Layout.jsx	Flex container with Sidebar + scrollable main content area
Sidebar.jsx	Navigation links with Lucide icons; Analysis route commented out for v3
LiveChart.jsx	uPlot integration: fixed X-axis grid, dynamic Y-axis auto-scaling, pause/resume with background buffer, 5-second rolling window, View Expanded button
Dashboard.jsx	CSS Grid widget layout; polls /api/system_status every 5 seconds; fetches device name and coordinates from /api/settings on mount
Settings.jsx	Three-tab interface (General / Network / System); React-Leaflet map; Wi-Fi nmcli integration; Data Sharing toggle; UDP target CRUD; Calibration save; Restart modal
Export.jsx	Timezone-aware datetime range pickers; CSV download via /api/export
Expanded.jsx	Full-screen chart page; hides sidebar, header, and View Expanded button; Pause/Resume only control
TimeZoneContext.jsx	Global timezone selector persisted to localStorage; available zones: UTC, US East/West, London, Tokyo, Colombo, Auckland

11. Quick Reference

11.1 Startup Checklist

1. Hardware assembled and wired per the GPIO pin map in Section 2.
2. Raspberry Pi OS Lite (64-bit) flashed and SSH enabled.
3. Hostname set to cl via raspi-config or hostnamectl.
4. SPI interface enabled via raspi-config → Interface Options → SPI.
5. Repository cloned to ~/EEW-Sensor-Firmware-and-UI.
6. Python virtual environment created and requirements installed.
7. Frontend built with npm run build.
8. Manual test passed: python main.py serves UI on port 8000.
9. setup_service.sh run successfully; service shows Active (running).
10. Dashboard accessible at http://cl.local from another device.

11.3 Default Ports and URLs

Service	Address
Dashboard (production, port 80)	http://cl.local or http://<pi-ip>
Dashboard (development Vite, port 5173)	http://localhost:5173
Backend API (development, port 8000)	http://localhost:8000
UDP data relay (default)	UDP port 2098 on configured target IPs

11.4 Configuration File Locations

Item	Location
Sensor settings (SQLite)	backend/eew_sensor.db — settings table
Sensor data (SQLite)	backend/eew_sensor.db — sensor_data table
systemd service file	/etc/systemd/system/eew-sensor.service
Python virtual environment	~/EEW-Sensor-Firmware-and-UI/.venv/
Hardware parameters	backend/sensor.py (lines 14–22)